Audio Guided Microbot

Documentation (Embedded Systems Workshop)

Team Sensodyne (#17) — Ayush, Pranav, Yashav, Raza
Under Prof. Abhishek Srivastava TA: Inesh Dheer

December 2, 2025

At a Glance (Our Project)

An autonomous microbot that listens for four simple voice commands (`forward, back, left, right`), moves safely, and can be taken over from a laptop with a joystick. **All recognition happens on the bot** (no internet). **Bluetooth (BLE) is our main, encrypted link**. LoRa is optional for long range. The bot stops if an obstacle is too close or if the link drops.

- **Why this is useful:** hands-free control, works offline and keeps audio private, easy supervision with a GUI.
- **Core pieces:** the mic (ears), a tiny on-device model (brain), motor driver + wheels (legs), ultrasonic sensor (safety), BLE (link), and a simple desktop GUI (eyes).

Contents

1 Problem & Our Approach **3**
1.1 Introduction . . . 3
1.2 Problem Statement & Constraints . . . 3
1.3 Objectives & Success Criteria . . . 4
1.4 Our Solution (high level) . . . 4

2 How the System Fits Together **5**

3 The Parts We Used (and why) **5**

4 The Brain: How We Understand the Voice **6**
4.1 From sound to something the bot can read . . . 6
4.2 What the tiny model does . . . 7
4.3 How we avoid mistakes in noisy rooms (practical guards) . . . 7

5 Communication & Security (practical view) **8**
5.1 Bluetooth Low Energy (BLE) — our main link . . . 8
5.2 LoRa — optional long-range . . . 8

6 What the GUI shows (operator view) **8**

7 What happens on each command (simple behavior) **9**

8 What We Calibrated & Tested **9**
8.1 Audio & model 9
8.2 Motion & safety 9
8.3 Battery life (4×AA (600mah each) 9
8.4 BLE and LoRa (field notes) 10

9 Results (short and clear) **10**

10 Challenges & What We Changed **10**

11 How to Run (In short) **10**

12 Implementation Details (how our code works, in simple words) **10**
12.1 Voice path: from mic to a decision 10
12.2 Command path: from a decision to motor motion 11
12.3 Safety logic (forward motion + link loss) 12
12.4 Where to tune (friendly knobs) 12
12.5 What runs in parallel (timing picture) 13
12.6 If something feels off (quick troubleshooting) 13

1 Problem & Our Approach

1.1 Introduction

Small robots are great for demos, labs, and quick tasks—but most voice control systems depend on the cloud, which adds delay and privacy issues. Our goal was to make a tiny microbot that you can talk to in plain words (`forward, back, left, right`), that reacts quickly, stays safe around obstacles, and can be taken over instantly from a laptop—*all while working fully offline.*

The design favors:

- **Simplicity for the operator:** four clear voice commands, one clean GUI, and an instant Manual Override.
- **Safety first:** obstacle halts and link-loss stop; no surprises in crowded rooms.
- **Privacy by default:** on-device recognition—no audio leaves the bot.
- **Practical hardware:** ESP32-S3 with built-in BLE, a single digital mic, and a basic differential drive.

1.2 Problem Statement & Constraints

We want a voice-driven microbot that is fast, safe, and private, using inexpensive parts. It should understand four commands, move appropriately, and be supervise-able from a laptop. No cloud is allowed.

Functional requirements

- Recognize `forward, back, left, right` on-device and act on them.
- Halt forward motion automatically when an obstacle is detected at 6 cm.
- Provide a **Manual Override** via GUI to drive with joystick/keys at any time.
- Keep the operator informed with simple BLE messages: `FORWARD, LEFT, OBSTACLE, IDLE`, etc.
- Stop safely if the operator link is lost (short timeout, ∼400 ms).

Non-functional requirements

- **Offline**: no internet; audio never leaves the device.
- **Low latency**: decision in well under 150 ms (we achieve ∼102 ms).
- **Small model**: must fit in microcontroller memory (we use ∼81 kB).
- **Low false triggers**: practical guards for noisy rooms (energy/ZCR gate, short memory, per-command confidence, cooldown).
- **Battery-friendly**: run on 4×AA cells for a couple of hours (measured 2.25 3.5 h depending on duty).

Scope & assumptions (hardware/environment)

- **Compute**: ESP32-S3; on-board BLE; FreeRTOS tasks.
- **Audio**: single INMP441 digital mic; 16 kHz sampling; ∼1 s analysis window.
- **Drive**: DRV8833 + 2×N20 motors (differential); ultrasonic sensor (HC-SR04) facing forward.
- **Typical spaces**: classrooms, labs, corridors; normal indoor noise.
- **Comms**: **BLE (primary, encrypted when paired)**; LoRa optional with app-level encryption when used.

1.3 Objectives & Success Criteria

- **Command responsiveness:** voice→action in ≤120 ms (*met*: ∼102 ms).
- **Safety halt:** forward motion stops at <6 cm and on link loss (*met*: periodic checks + timeout stop).
- **Operator clarity:** GUI mirrors state with simple words; Manual Override is instant.
- **Robustness in noise:** few/no unintended moves in typical indoor noise thanks to gating + thresholds + cooldown.
- **Runtime on 4×AA:** 2.25 −3.5 h depending on duty (*measured*, see §8.3).

1.4 Our Solution (high level)

- **Listen & understand:** a compact model on the ESP32-S3 recognizes `forward/back/left/right`.
- **Move safely:** before and during forward motion, we check distance in front; halt at 6 cm.
- **Stay connected: BLE is primary (encrypted by default when paired)**; optional LoRa adds range with an extra app-level lock (encryption) on messages.
- **Be easy to control:** a desktop GUI shows status and lets the operator drive instantly with a joystick.

2 How the System Fits Together

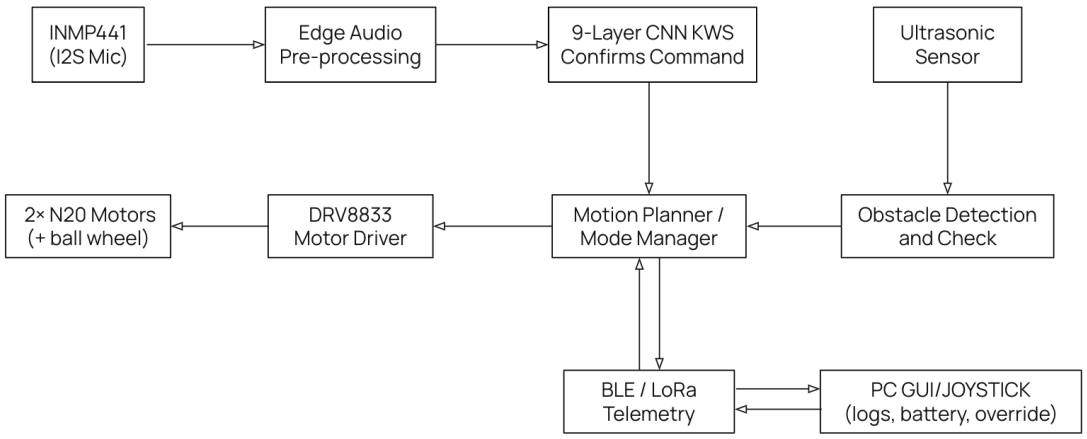

Figure 1: Mic (hear) → tiny model (decide) → safety check (stop if needed) → wheels (act). BLE links the bot to the GUI.

Simple flow Mic hears the word → we turn the sound into a small picture-like grid → the tiny model picks the best of four commands → a few "common-sense" checks avoid mistakes → motors run or stop → GUI shows what's happening and can take over.

3 The Parts We Used (and why)

- **ESP32-S3 board (brain):** enough memory to run our tiny model on-device; built-in BLE keeps wiring simple.
- **INMP441 microphone (ears):** clean digital mic input.
- **HC-SR04 ultrasonic (safety):** halts forward motion within a set distance (default 6 cm).
- **DRV8833 + 2×N20 motors (legs):** sturdy differential drive; easy to control with four pins (`IN1..IN4`).
- **Power:** 4×AA cells with regulation; we measured runtime at different speeds (see §8.3).
- **Shell & PCB:** small 3D-printed cover and a simple PCB to avoid loose wires and reduce noise.

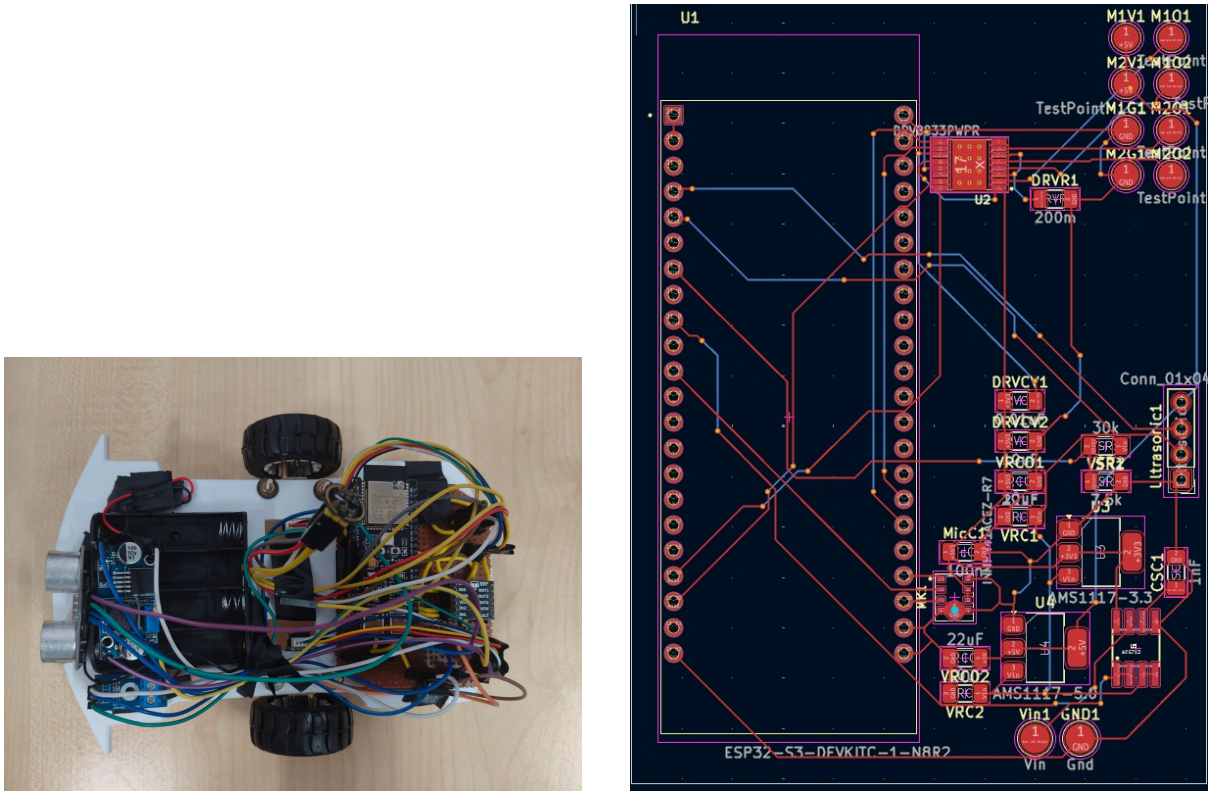

Figure 2: Hardware and PCB

4 The Brain: How We Understand the Voice

4.1 From sound to something the bot can read

We convert a short slice of audio (about 1 s) into a small grid (time vs. pitch). It's like a simple black-and-white image of the word. Words such as `left` and `right` form slightly different shapes on this grid.

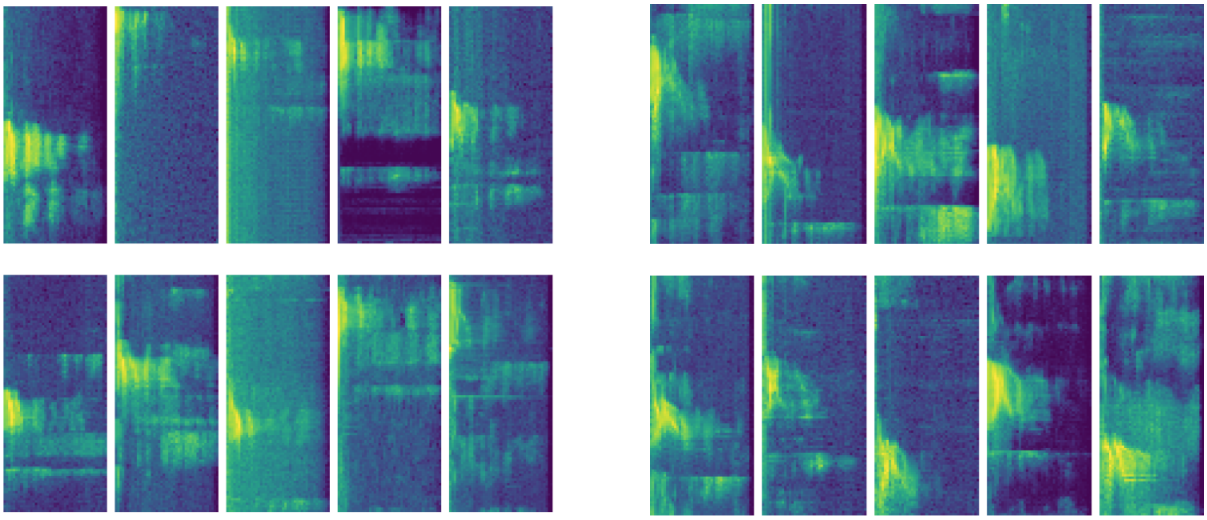

Figure 3: Example "voice pictures" for `left` and `right`.

4.2 What the tiny model does

- A very small CNN looks for patterns in those grids.
- **Size on device:** ∼81 kB; **decision time:** ∼102 ms.
- **Why this matters:** quick reactions, fits in memory, fully offline (no privacy worries).

4.3 How we avoid mistakes in noisy rooms (practical guards)

We layer a few friendly checks around the model so the bot doesn't move because of coughs or background talk.

1. **Only listen when speech is present (a simple gate):** we look at loudness, steadiness, and how often the signal crosses zero. If it looks like real speech, we "open the gate" and run the model; otherwise we skip it and treat it as silence.
2. **Confidence per word:** each command has its own "confidence to accept" and a "margin" over the second best. Clear words pass easily; unclear ones are ignored.
3. **Short memory:** we keep a tiny rolling window of the last few model outputs and accept only if the same command stays on top—this avoids one-frame spikes.
4. **Cooldown:** after acting on a word, we wait a bit before accepting the same word again—helps with echoes.

Quick reference (from firmware)

All of this is already coded; here are the friendly names and values your teammates will tweak most often.

Table 1: Speech gate (when to even run the model)

What it checks	Current setting
Needs to be louder than recent background (SNR)	normal `1.25`, "strong" `2.15`
Minimum steady loudness (RMS)	normal `58.5`, "strong" `106.5`
Extra margin above average noise (abs delta)	`6.5`
How busy the signal is (zero-crossing rate)	must be ≤ noise+margin; cap `0.28` (+`0.08` marg
Open gate after this many "hits"	`2` (keep open briefly for up to `600` ms)

Table 2: Accepting a command (per-word confidence)

Command	Min confidence	Extra margin over 2nd
`forward`	`0.70`	`0.35`
`backward`	`0.71`	`0.37`
`left`	`0.50`	`0.18`
`right`	`0.48`	`0.15`
`(unknown/silence)`	`0.50`	`0.20`

Extra: when speech is very strong, the code can relax these slightly (to `0.45/0.15`) so good words trigger faster. Cooldown between accepted commands: **1,000 ms**. Debug status prints every **200 ms**.

5 Communication & Security (practical view)

5.1 Bluetooth Low Energy (BLE) — our main link

BLE is the default. It's quick, stable indoors, and **already encrypted** once paired (we use normal paired operation). We send short text updates like `FORWARD`, `LEFT`, `OBSTACLE`, `IDLE` so the GUI mirrors the bot's state. If packets stop for a short time, the bot stops.

5.2 LoRa — optional long-range

When we need distance, we can switch to LoRa. Because LoRa itself isn't end-to-end encrypted, we add a simple *lock on the message* (shared secret → encrypted payload) so packets stay private and tamper-evident.

6 What the GUI shows (operator view)

- **One window** to connect, see logs, battery and motion state.
- **Manual Override** button: instantly take control with keys or a joystick; the bot switches out of voice mode.
- **Safety:** if BLE packets stop arriving for a short time (∼400 ms), the bot auto-stops.

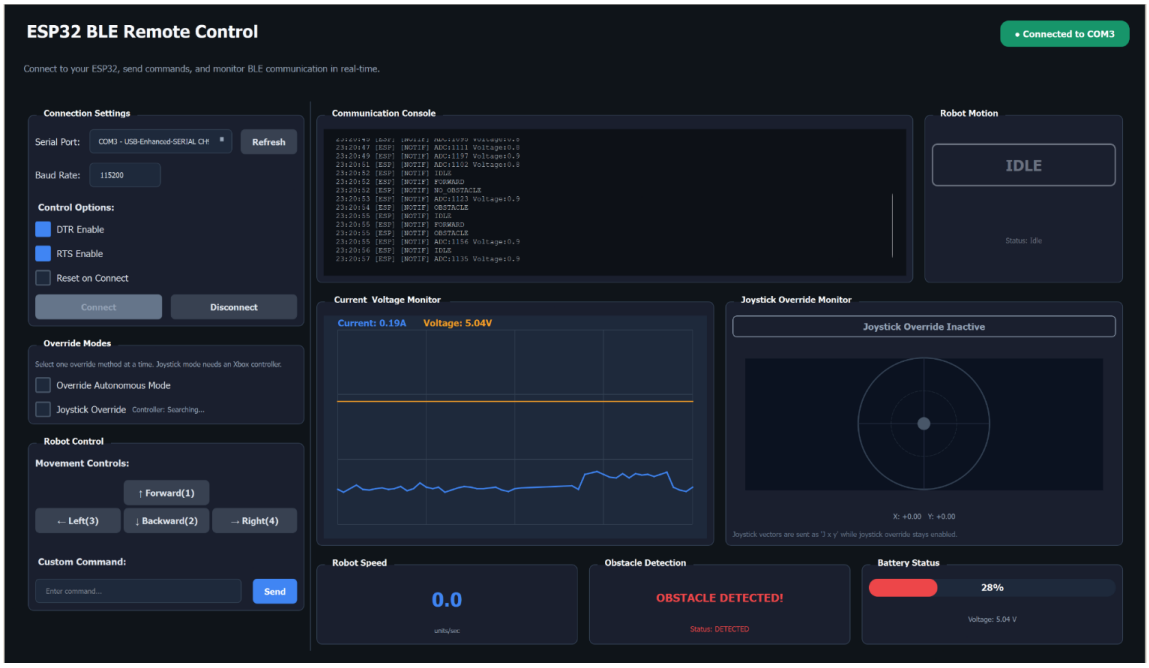

Figure 4: Remote GUI: status, logs, and joystick drive.

7 What happens on each command (simple behavior)

Below is how the bot moves after a command is accepted. We also send a short BLE message for the GUI.

Table 3: Command behavior (summarized from firmware)

Command	Safety check	Motion pins (IN1..IN4)	Typical durat
`forward`	Check distance each step; stop < 6 cm	`1:HIGH,2:LOW,3:LOW,4:HIGH`	up to $\sim$9 s (rec
`backward`	No front check needed	`1:LOW,2:HIGH,3:HIGH,4:LOW`	$\sim$5 s
`left`	No front check needed	`1:HIGH,2:LOW,3:HIGH,4:LOW`	$\sim$5 s (pivot)
`right`	No front check needed	`1:LOW,2:HIGH,3:LOW,4:HIGH`	$\sim$5 s (pivot)

At the end of any move, we set all four pins LOW, send `IDLE`, and disable the motor driver to save power.

Front distance sensor (forward only) We trigger the ultrasonic sensor, read the echo, and compute distance. If it's valid and < 6 cm, we stop immediately; otherwise we continue but recheck frequently.

8 What We Calibrated & Tested

8.1 Audio & model

- Trim clips to keep speech; keep lengths consistent so the "voice picture" is stable.
- Tune confidence thresholds (per word) and cooldown until the bot feels both responsive and safe.
- In busy spaces, the speech gate + "short memory" window keep false triggers rare.

8.2 Motion & safety

- Map joystick values to left/right wheel speeds; add a small left/right correction so "straight" is truly straight.
- Verify the 6 cm stop distance with a ruler and adjust only if needed.

8.3 Battery life (4×AA (600mah each)

Table 4: Runtime vs. movement (measured)

Motor duty	Avg current (A)	Runtime (h)	hh:mm
0%	0.18	$\sim$3.50	$\sim$3:30
25%	0.198	$\sim$3.00	$\sim$3:00
50%	0.215	$\sim$2.75	$\sim$2:45
75%	0.232	$\sim$2.50	$\sim$2:30
100%	0.250	$\sim$2.25	$\sim$2:15

8.4 BLE and LoRa (field notes)

Indoors, BLE is strong across rooms and a couple of walls; very long hallways or lots of motion can weaken it. LoRa reaches far in open sight; when we use it, we keep our app-level lock (encryption) on messages.

9 Results (short and clear)

- **On-device voice:** ∼102 ms decision time; smooth in quiet rooms; usable in busy rooms thanks to the guards.
- **Safety working:** stops within 6 cm and also on link loss.
- **Operator experience:** joystick feels immediate; GUI is clear; manual override is instant.

10 Challenges & What We Changed

- Early plan used multiple mics to find direction—hardware wasn't precise enough, so we pivoted to solid keyword control.
- A browser model worked online but not on the board; switching to a direct `.tflite` with a tiny runtime fixed it.
- Noise from wiring/breadboards caused false triggers; a PCB and the extra guards stabilized behavior.

11 How to Run (In short)

1. Flash the ESP32-S3 with the firmware that includes the tiny model file (`.tflite`).
2. Power the bot; pair over BLE from the GUI.
3. Choose **Voice** mode to use keywords, or tap **Manual Override** for joystick/keys.
4. Keep obstacles $\geq$ 6 cm in front when moving forward; if the link drops, the bot will stop on its own.

12 Implementation Details (how our code works, in simple words)

This section explains *what actually runs on the board* and how each piece talks to the next. We keep it practical and match the names you will see in code and logs.

12.1 Voice path: from mic to a decision

1. **Keep the last ∼1 second of audio in memory** We continuously sample the mic and store it in a ring buffer (`AUDIO_LENGTH=16000` samples). *Why:* when a word is spoken, we already have the recent audio to analyze.

2. **Decide if this even sounds like speech ("open the gate")** Before running the tiny model, we check simple metrics over the last second: average loudness, steadiness (RMS), and how often the signal crosses zero (ZCR). If it looks like real speech (loud enough, steady enough, and not too "buzzy"), we **open the gate**; otherwise we skip this turn. *Names in code:* `shouldRunInference(...)` with thresholds like SNR `1.25 / 2.15`, RMS `58.5 / 106.5`, max ZCR `0.28` (+`0.08` margin), min hits `2`, release `600 ms`. *Why:* saves time and avoids reacting to coughs and background chatter.

3. **Turn audio into a small "voice picture"** With the gate open, we build a compact spectrogram (a time–pitch grid). *Name in code:* `AudioProcessor::get_spectrogram(...)`.

4. **Run the tiny model on that picture** The model returns 5 scores: `forward, backward, left, right, (unknown/silence)`. *Name in code:* `m_nn->predict(); m_nn->getOutputBuffer()`.

5. **Keep a short memory of recent outputs** We store each new set of scores in a rolling window (`COMMAND_WINDOW` rows). Internally we keep them as *log* values so combining them stays numerically stable. *Name in code:* `cachePredictionRow(...)` updates `m_scores[window][class]`.

6. **Pick the best command, check confidence and margin** We add up the windowed scores per class, compute a best class and a runner-up, then require: (a) **Min confidence** for that command, and (b) **extra margin** over the second place. *Per-command thresholds in code:*

   - `forward` → 0.70 (min), 0.35 (margin)
   - `backward` → 0.71, 0.37
   - `left` → 0.50, 0.18
   - `right` → 0.48, 0.15
   - `unknown/silence` → 0.50, 0.20

   When the speech looks very strong, the code can relax the generic thresholds slightly (to ∼0.45/0.15) so good words trigger faster. *Name in code:* see `kCommandProbabilityThresholds`, `kCommandProbabilityMargins` and the `accept_command` checks.

7. **Cooldown so echoes don't double-trigger** After we act on a word, we wait `1000 ms` before we accept the *same* word again. *Name in code:* `kDetectionCooldownMs = 1000`.

Debugging while you test Every ∼200 ms we print a compact line showing gate status (loudness, ZCR, SNR), top command, probabilities, thresholds, and whether we accepted it. *Name in code:* `kDebugLogIntervalMs = 200`. This is the first place to look if the bot is too jumpy or too sleepy.

12.2 Command path: from a decision to motor motion

Once a command is accepted, we don't move motors directly from the detector thread. Instead, we **queue** the command, and a separate task performs the action. *Why:* keeps the audio/model loop smooth and responsive.

1. **Queue the command** `queueCommand(index, best_score)` pushes into a FreeRTOS queue (`xQueueSendToBack`).
2. **A dedicated task executes it** `commandQueueProcessorTask(...)` waits on the queue and calls `processCommand(index)`.
3. **What each command does (summarized)**
   - **forward** Before moving, we ping the ultrasonic sensor and compute distance. If it's valid and < 6 cm, we report `OBSTACLE` and do not move. Otherwise we enable the driver (`DRV_EEP = HIGH`), set motor pins to forward (`IN1=HIGH, IN2=LOW, IN3=LOW, IN4=HIGH`) and *keep rechecking distance* roughly every 200 ms in a short loop. If at any step an obstacle comes within 6 cm, we stop immediately and report `OBSTACLE`.
   - **backward** Enable driver and set reverse pins (`IN1=LOW, IN2=HIGH, IN3=HIGH, IN4=LOW`) for a brief, fixed duration.
   - **left/right** Pivot in place by driving wheels in opposite directions. Left: (`IN1=HIGH, IN2=LOW, IN3=HIGH, IN4=LOW`). Right: (`IN1=LOW, IN2=HIGH, IN3=LOW, IN4=HIGH`).
4. **Always finish safely** After any command completes (or is interrupted by an obstacle), we: set `IN1..IN4 = LOW`, send `IDLE` over BLE, and disable the motor driver (`DRV_EEP = LOW`) to save power.

BLE messages you'll see in the GUI On each action we notify short, human-readable strings: `FORWARD, BACKWARD, LEFT, RIGHT, OBSTACLE, NO_OBSTACLE, IDLE`. These make it obvious what the bot is doing and why it stopped.

12.3 Safety logic (forward motion + link loss)

- **Obstacle stop (forward only)** The distance check runs *before* moving and *during* the forward run (periodic pings). If distance < 6 cm at any instant, we stop and show `OBSTACLE`.
- **If the operator link drops** The GUI sends frequent small updates. If those stop for a short window ($\sim$400 ms), the bot falls back to `IDLE`/stop. *Tip:* if you're testing without the GUI, keep voice mode enabled so the safety still holds.

12.4 Where to tune (friendly knobs)

- **How eager the bot is to treat audio as speech:** SNR/RMS/ZCR thresholds ("gate"). If it's too jumpy, raise SNR/RMS or tighten ZCR. If it's too sleepy, lower them slightly.
- **How strict a word must be:** per-command confidence and margin. If "right" is mis-heard, raise its threshold or margin a little.
- **Repeat protection:** cooldown (`1000 ms`). Increase for echoey rooms; decrease if you want rapid repeated turns.

12.5 What runs in parallel (timing picture)

- The **detector loop** samples audio, opens/closes the speech gate, builds the voice picture, runs the model, and decides on commands. It keeps the UI responsive by *not* touching motors directly.
- The **command task** takes accepted commands from the queue and handles motors + safety checks.
- The **BLE/GUI loop** shows state and allows instant Manual Override (it simply bypasses voice and drives the same motor pins).

12.6 If something feels off (quick troubleshooting)

- **Bot triggers on noise:** raise SNR/RMS gate; raise per-command confidence; check wiring/PCB ground.
- **Bot misses clear words:** slightly lower per-command threshold; reduce margin; ensure the mic opening is unobstructed.
- **Forward keeps stopping:** confirm ultrasonic wiring; measure if your desk is within 6 cm; try temporarily raising the stop distance to test.
- **GUI shows no updates:** re-pair BLE; ensure notifications are enabled; watch serial logs for `IDLE/OBSTACLE`.

Appendix A — Parameters (from firmware)

- **Audio slice length:** `AUDIO_LENGTH` = `16000` samples (≈ 1 s at 16 kHz).
- **Window & step:** `WINDOW_SIZE` = `320` (≈ 20 ms), `STEP_SIZE` = `160` (≈ 10 ms hop).
- **Pooling:** `POOLING_SIZE` = `6` (compact "voice picture").
- **Gate thresholds:** SNR `1.25`/`2.15`, RMS `58.5`/`106.5`, max ZCR `0.28` (+`0.08`), min hits `2`, hangover hits `4`, release `600 ms`.
- **Per-word acceptance:** see table in § *The Brain*.
- **Cooldown:** `1000 ms`; **Debug print interval:** `200 ms`.
- **BLE messages:** `"FORWARD"`, `"BACKWARD"`, `"LEFT"`, `"RIGHT"`, `"OBSTACLE"`, `"NO_OBSTACLE"`, `"IDLE"`.
- **Ultrasonic pins (example in code):** TRIG `GPIO 4`, ECHO `GPIO 5` (adjust if your wiring differs).

Appendix B — Figures added from the build

- `figures/block-diagram.png` (single architecture diagram)
- `figures/hardware.jpg` (top view of the bot)
- `figures/pcb.jpg` (assembled PCB)

- `figures/spectrogram-left.png`, `figures/spectrogram-right.png`
- `figures/gui.png` (GUI screenshot)